

PART 3 OF 3

Harness

Security Architecture, Supply Chain Assurance, Secrets Management,
and Observability / DORA Metrics

Continuation of the earlier architecture and AI-agent reports · Research date: July 2026

1. Supply Chain Security (SCS / SSCA)

Harness's supply chain module (branded SCS, originally launched as SSCA — Software Supply Chain Assurance) is built around three artifacts that travel with every build: an **SBOM** (what's inside), **SLSA provenance** (how and where it was built), and a cryptographic **attestation** binding the two together so downstream systems can verify neither has been tampered with. **DOCUMENTED**

How attestation actually works

```
Build → SBOM/SLSA provenance generated → Cosign signs it → → Keyless: OIDC
identity → Fulcio issues short-lived cert → sign → discard key → Key-based: user-
managed key pair (HashiCorp Vault only, as of current docs) → Signed .att file
pushed to container registry, tied to image digest → (optional) event logged to
Sigstore's public Rekor transparency log → Downstream consumer verifies signature
against trusted root before deploy/promote
```

The keyless path is the operationally important one: it eliminates long-lived private key custody by using workload identity (OIDC) to obtain a short-lived signing certificate at build time, which Cosign uses and immediately discards. This follows the standard Sigstore toolchain (Fulcio for certs, Cosign for signing, Rekor for the transparency log) that has become the de facto industry standard for artifact signing.

DOCUMENTED

SLSA compliance levels

Level	Requirement	Harness support
Level 1	Provenance exists — documents builder, process, top-level inputs	Supported on SaaS and Self-Managed (SMP)
Level 2	Adds: build runs on a hosted platform that generates and signs provenance itself	Supported on SaaS and SMP
Level 3	Adds: isolated, ephemeral build infrastructure (new infra per run, destroyed after) + non-privileged, no-volume-mount build steps so provenance-signing keys are unreachable even by the build script itself	SaaS-hosted builds only — not achievable in Self-Managed Enterprise Edition, because it requires Harness-hosted, per-run ephemeral build infrastructure

Source: Harness SCS documentation. **DOCUMENTED** — this is a genuinely useful diligence item: an organization requiring SLSA Level 3 as a compliance bar cannot get there on a fully self-managed/air-gapped Harness deployment.

Air-gapped caveats worth flagging to security reviewers

- Repository Security Posture Management is **not supported** in air-gapped environments.
- SBOM license data for some dependencies falls back to "NOASSERTION" in air-gapped mode, reducing SBOM quality score (does not affect generation itself).

- Rekor public transparency-log logging is skipped in air-gapped mode (does not affect the attestation's validity, just its public visibility).

Policy enforcement runs through the same OPA engine covered in the architecture report: SBOM-based policies can block a build based on component name, license, or package URL (PURL), and a "zero-day remediation" workflow lets teams search for a newly-disclosed CVE across all SBOMs and block it in the next build via policy — without waiting for a full re-scan cycle. **DOCUMENTED**

2. Secrets Management

Harness ships a built-in secret manager but is designed to defer to external ones — AWS Secrets Manager, Azure Key Vault, GCP Secret Manager, HashiCorp Vault, and CyberArk Conjur (via custom secret manager) are all first-class connector types, and the docs explicitly recommend disabling the built-in manager once a real one is wired up. **DOCUMENTED**

Notable operational detail on the Vault integration: Harness supports AWS Auth to Vault via IAM principals or EC2 instance identity, meaning the Delegate can retrieve a Vault token without ever handling a static Vault credential — the recommended pattern specifically because it avoids managing long-lived tokens, usernames, or passwords by hand.

Industry context worth noting: Vault itself is widely regarded in 2026 as the most feature-complete secrets platform for high-security environments (dynamic secrets, encryption-as-a-service, PKI, SSH credential brokering) but also the one with the highest operational overhead — several 2026 comparison sources flag it as requiring genuine platform-engineering capacity to run well, versus lighter developer-first tools (Doppler, Infisical) that suit less regulated teams. This is a general secrets-management market observation, not Harness-specific, but relevant when an enterprise is deciding whether to pair Harness with Vault or a lighter-weight secrets backend. **ANALYSIS**

3. Observability: DORA Metrics and Software Engineering Insights

Harness tracks the four (now sometimes five, adding Reliability/MTBF) DORA metrics — Deployment Frequency, Lead Time for Changes, Change Failure Rate, and Mean Time to Restore — at **two distinct layers**, and the distinction matters for what each one can actually tell you:

Layer	Scope	Best for
CD DORA dashboard	Pipeline-level, Harness deployment activity only	Fast visibility into deployment health without wiring up anything else
Software Engineering Insights (SEI)	Organization-wide, correlates 40+ integrated data sources (Jira, GitHub, GitLab, Jenkins, PagerDuty, etc.) against a configurable DORA Profile and correlation engine	Root-causing <i>why</i> lead time is long — e.g., surfacing that delays trace to code-review bottlenecks in GitHub rather than anything in the CD pipeline itself

The correlation engine is the differentiated piece: rather than just reporting four numbers, SEI is explicitly built to trace a metric back to its upstream cause across tools Harness doesn't own — flaky Jenkins tests driving change failure rate, GitHub review latency driving lead time, and so on. **DOCUMENTED** — note this SEI capability originates from Harness's 2022 acquisition of Propelo (visible in the product's internal URL namespace, `propelo-sei`), a useful data point if evaluating maturity/integration depth of this specific module versus newer-built ones.

4. How This Connects Back to the Earlier Reports

- **To the architecture report:** supply-chain policy enforcement and DORA correlation both run through the same OPA-based governance model and the same Delegate execution boundary already covered — this isn't a bolted-on security product, it's the same control plane applying policy to a different entity type.
- **To the AI-agent report:** a Worker Agent's own actions (e.g., an Autofix or IaCM Remediation agent committing a change) are auditable through the same DORA/SEI correlation pipeline as a human-triggered deployment — meaning "did agent-generated changes increase our change failure rate?" is, in principle, an answerable SEI query rather than a blind spot. This wasn't independently verified in current research and is worth confirming directly if agentic change volume is expected to be material.

ANALYSIS

This closes out the three-part scoped treatment (Architecture → AI Agents & Vendor Landscape → Security/Supply Chain/Observability) of the original 23-section research prompt. Remaining untouched sections — per-industry reference architectures, the 15 hands-on labs, sequence diagrams, and interview question banks — are lower-leverage for an architect's decision-making and are better suited to the Research feature if still wanted.